

07_Gradient Descent

Suppose your linear regression model is: $\hat{y} = w^T x + b$

Your loss function is typically **MSE**: $J(w, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$

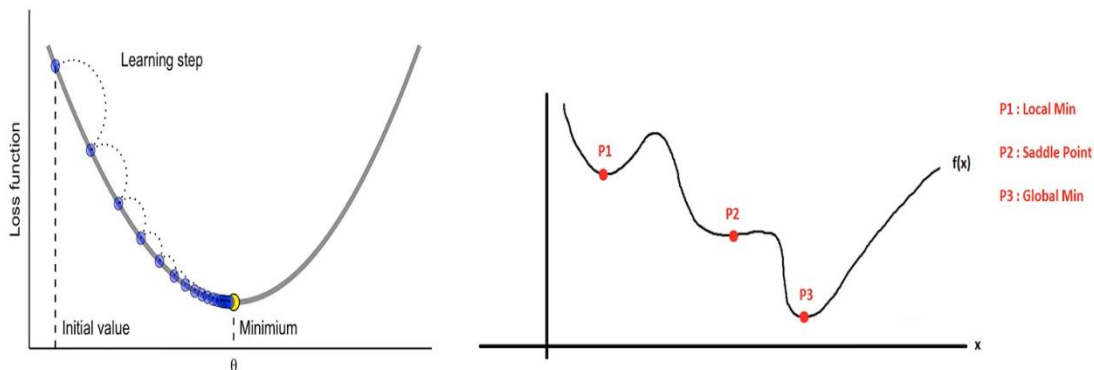
This cost function is **convex**, so gradient descent will converge to the **global minimum**

1. Compute the gradients: $\frac{\partial J}{\partial w} = \frac{2}{n} X^T (Xw + b - y)$

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

2. Update the parameters: $w := w - \alpha \frac{\partial J}{\partial w}$ $b := b - \alpha \frac{\partial J}{\partial b}$

Gradient descent is a first-order **iterative optimization algorithm** for finding the minimum of a function. Gradient descent can be performed on any differentiable loss function. The main goal of gradient descent is to minimize a cost or loss function, optimizing model parameters for better performance.



Convexity doesn't depend on MSE alone; It depends on the combination of the cost function and the model.

The main equation for updating parameters in gradient descent is:

$$x_{\text{next}} = x - \alpha \cdot \nabla f(x) \text{ or, } w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial J}{\partial w}$$

where:

- x_{next} is the updated parameter value,
- x is the current parameter value,
- α is the learning rate,
- $\nabla f(x)$ is the gradient of the function f at x .

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial J}{\partial w}$$

where:

- w_{new} is the updated weight.
- w_{old} is the current weight.
- α is the learning rate.
- $\frac{\partial J}{\partial w}$ is the gradient of the loss function with respect to the weight.

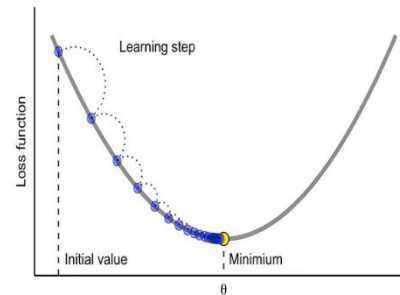


Fig: Gradient Decent

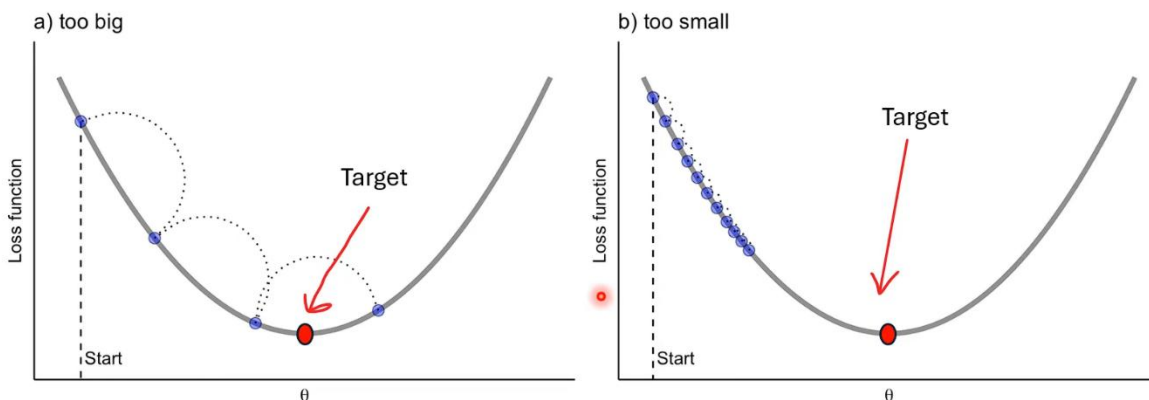
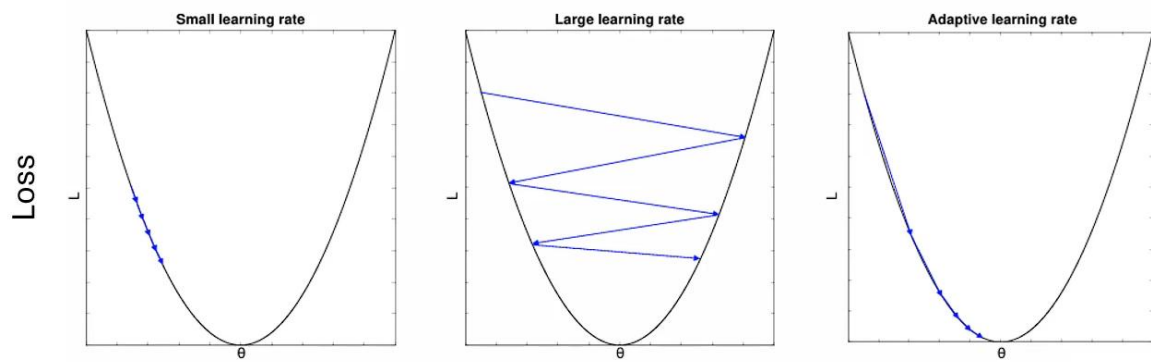


Fig: Importance of Learning Rate



- η too small: long training time
- η too large: miss optima
- Practice: “learning rate decay”: adapt η gradually (e.g.: start with $\eta = 0.01$ and divide every x epoch by 10)

Momentum & NAG With Gradient Descent

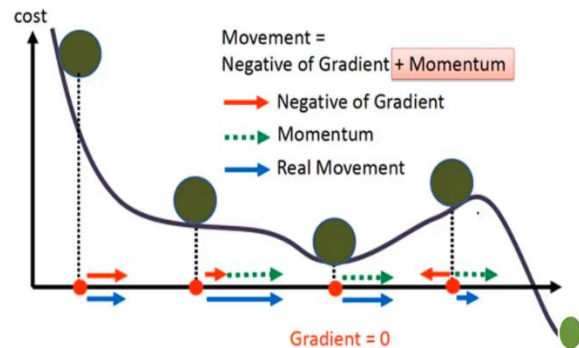
Gradient Descent With Momentum

Momentum: Momentum is a technique used to accelerate gradient descent algorithms. It improves the stability and speed of convergence in gradient-based optimization by combining the current gradient with the past gradients' influence, leading to more efficient training of machine learning models.

Momentum helps the optimization process by considering both the current gradient and an accumulation of past gradients. There are **two main** steps involved in applying momentum to the gradient descent optimization process:

1. Velocity Update
2. Parameter Update

Note: The main point of the diagram is to illustrate how momentum helps to avoid getting stuck in local minima and can push the optimization process past these points, unlike traditional gradient descent which might oscillate or come back to a local minimum without momentum.



- ❖ The **red** arrows indicate the direction of the negative gradient. If the algorithm relied only on these, it might get stuck in local minima where the gradient is zero.
- ❖ The **green** dotted arrows represent the momentum. This term accumulates the gradients over time and can help in moving past small local minima.
- ❖ The **blue** arrows show the real movement of the parameters, which is the combined effect of the negative gradient and the momentum.

1. Velocity Update:

$$\mathbf{v}^{(k)} = \mu \mathbf{v}^{(k-1)} - \eta \nabla L(\mathbf{w}^{(k)})$$

- $\mathbf{v}^{(k)}$ represents the velocity or the accumulated gradient at iteration k .
- μ is the momentum coefficient.
- η is the learning rate.
- $\nabla L(\mathbf{w}^{(k)})$ is the gradient of the loss function with respect to the weights $\mathbf{w}$ at iteration k .
- $\mu \mathbf{v}^{(k-1)}$: This term incorporates the influence of past gradients. The coefficient μ (usually between 0.9 and 0.99) determines the contribution of the previous velocity. Higher μ values give more weight to the accumulated past gradients.
- $-\eta \nabla L(\mathbf{w}^{(k)})$: This term represents the current gradient scaled by the learning rate η . This influences the direction and magnitude of the update based on the current gradient.

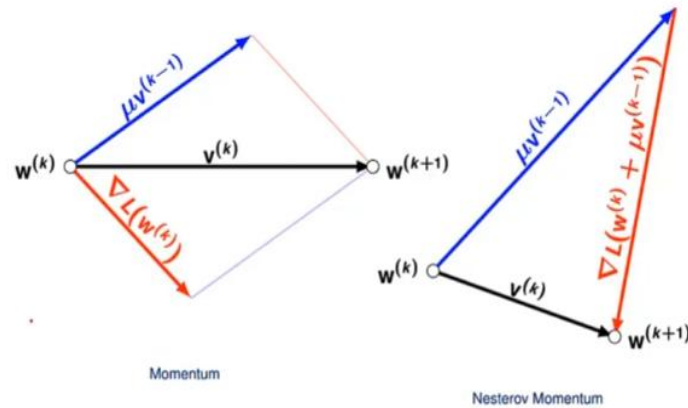
Parameter Update:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \mathbf{v}^{(k)}$$

- $\mathbf{w}^{(k+1)}$: The new parameters for the next iteration.
- $\mathbf{w}^{(k)}$: The current parameters.
- $\mathbf{v}^{(k)}$: The updated velocity that combines the past and current gradient information.

These two steps together ensure that the parameter updates benefit from both the current gradient and an accumulated direction of previous gradients, which helps to smooth out the path towards the minimum of the loss function and accelerates convergence.

Nesterov Accelerated Gradient (NAG) / Nesterov Momentum in ML



- **Traditional Momentum:** Uses the current gradient at the current position $\mathbf{w}^{(k)}$ along with an accumulated momentum term derived from past gradients.
- **Nesterov Momentum:** Computes the gradient at a future anticipated position $\mathbf{w}^{(k)} + \mu\mathbf{v}^{(k-1)}$, providing a more forward-looking approach that can lead to better convergence properties.
- **Works on future position:**
 - In Nesterov momentum, the gradient is computed at a lookahead position, which takes into account where the parameters will be after the current step.
 - The update equations are:

$$\mathbf{v}^{(k)} = \mu\mathbf{v}^{(k-1)} - \eta\nabla L(\mathbf{w}^{(k)} + \mu\mathbf{v}^{(k-1)})$$

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \mathbf{v}^{(k)}$$

- The gradient $\nabla L(\mathbf{w}^{(k)} + \mu\mathbf{v}^{(k-1)})$ is computed at the anticipated future position $\mathbf{w}^{(k)} + \mu\mathbf{v}^{(k-1)}$. [Explanation in Next Slide >>](#)

Two main steps in Nesterov Momentum:

1. Velocity Update:

$$\mathbf{v}^{(k)} = \mu \mathbf{v}^{(k-1)} - \eta \nabla L(\mathbf{w}^{(k)} + \mu \mathbf{v}^{(k-1)})$$

- **Momentum Component:** $\mu \mathbf{v}^{(k-1)}$ is the contribution from the past velocity, scaled by the momentum coefficient μ .
- **Gradient Component:** $\eta \nabla L(\mathbf{w}^{(k)} + \mu \mathbf{v}^{(k-1)})$ is the learning rate times the gradient of the loss function, evaluated at the "lookahead" position $\mathbf{w}^{(k)} + \mu \mathbf{v}^{(k-1)}$.

2. Parameter Update:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \mathbf{v}^{(k)}$$

- The current parameters $\mathbf{w}^{(k)}$ are updated by adding the velocity $\mathbf{v}^{(k)}$.

1. Velocity Update:

$$\mathbf{v}^{(k)} = \mu \mathbf{v}^{(k-1)} - \eta \nabla L(\mathbf{w}^{(k)} + \mu \mathbf{v}^{(k-1)})$$

- $\mathbf{v}^{(k)}$: The velocity (momentum term) at iteration k .
- μ : The momentum coefficient, a value between 0 and 1 that determines the influence of the previous velocity.
- η : The learning rate, a small positive value that controls the step size during gradient descent.
- $\nabla L(\mathbf{w}^{(k)} + \mu \mathbf{v}^{(k-1)})$: The gradient of the loss function L evaluated at the "lookahead" position $\mathbf{w}^{(k)} + \mu \mathbf{v}^{(k-1)}$.
 - $\mathbf{w}^{(k)}$: The current parameters at iteration k .
 - $\mathbf{v}^{(k-1)}$: The velocity (momentum term) from the previous iteration.

2. Parameter Update:

$$\underline{\mathbf{w}^{(k+1)}} = \mathbf{w}^{(k)} + \mathbf{v}^{(k)}$$

- $\mathbf{w}^{(k+1)}$: The updated parameters for the next iteration.
- $\mathbf{w}^{(k)}$: The current parameters.
- $\mathbf{v}^{(k)}$: The updated velocity from the first equation.

Advantage of Nesterov Momentum:

- ❖ **Faster Convergence:** By computing the gradient at a future position, Nesterov momentum often leads to faster convergence and more efficient optimization.
- ❖ **More Accurate Updates:** The lookahead mechanism provides a more accurate correction to the parameter updates, which can lead to better performance, particularly in cases where the optimization landscape has high curvature or when dealing with noisy gradients.
- ❖ **Convex Optimization:** In convex optimization problems, Nesterov Momentum can significantly speed up convergence compared to standard gradient descent and even traditional momentum methods.
- ❖ **For Non-Convex Problems:** While lacking the same level of theoretical guarantees, it still provides practical benefits in many real-world applications, especially in deep learning and other complex optimization tasks.